

Actividades previas

De acuerdo con la principal actividad que le han delegado desde el área de ingeniería biomédica, donde es necesario verificar y actualizar las hojas de vida de los diferentes equipos biomédicos con los que cuenta la institución, es claro que se necesita trabajar en el diseño y las principales variables que el sistema de consulta y/o información debe tener. Sin embargo, antes de comenzar con estas actividades previas es importante dar respuesta desde la parte de gestión clínica a las siguientes inquietudes:

- ¿Qué normatividad está relacionada directamente con el desarrollo de las hojas de vida de los equipos biomédicos?
- ¿Cuáles son los principales campos que debe contener una hoja de vida para equipos biomédicos?
- ¿Qué tipos de registros o documentos anexos deberían tener la trazabilidad de un equipo biomédico a partir de su hoja de vida?

Finalmente, es clave determinar cuál sería la incidencia en términos de riesgos desde el punto de vista del uso continuo de la tecnología al no tener la hoja de vida de un equipo biomédico y sus posibles consecuencias frente al personal médico y los pacientes.

Para ello se sugiere que hagan uso de la metodología Failure Modes and Effects Critical Analysis (FMECA) para el desarrollo de un buen análisis de riesgos y con ello entregar argumentos sólidos que sustenten la necesidad de tener los documentos al día sobre el equipamiento biomédico.

Variables en VBA

Las variables en VBA son utilizadas para guardar valores y su tipo dependerá de la clase de datos que se desee guardar en cada una de ellas. En VBA existen variables de tipo entero, que almacenan valores o números enteros, que van desde los -32.768 a 32.767 si llegara a ser mayor o menor a este rango de valores, puede darse el caso que genere un error por desbordamiento.

La variable de tipo double que almacenan números decimales y aunque puede almacenar valores enteros, no es recomendable hacerlo debido a que se estaría desperdiciando espacio en la memoria del ordenador.

variables de tipo texto que almacenan una serie de cadenas de caracteres entre otros tipos de variables más. Esta variable se declara con la palabra reservada como "string" y puede contener valores alfanuméricos solo si no se necesita realizar operaciones matemáticas.

Una variable tipo boolean o lógica es la que tiene menos espacio ocupa en memoria ya que solo almacena si un valor es "verdadero" o "falso".

Las variables de tipo Date o fecha almacenan datos que conservan el formato de fecha.

Es importante tener en cuenta que a la hora de declarar una variable se debe hacer con la palabra clave reservada DIM y seguido el nombre que se le va a asignar a la variable.

Esta palabra reservada debe tener ciertas condiciones como:

- No debe tener espacios entre palabras
- No empezar por caracteres numéricos
- No utilizar caracteres especiales a diferencia del guion bajo
- El nombre debe ser acorde a lo que se va a almacenar
- La primera letra debe empezar con mayúscula
- La cantidad máxima de caracteres es de 30

Seguido de estas recomendaciones debe ir la palabra **As**:

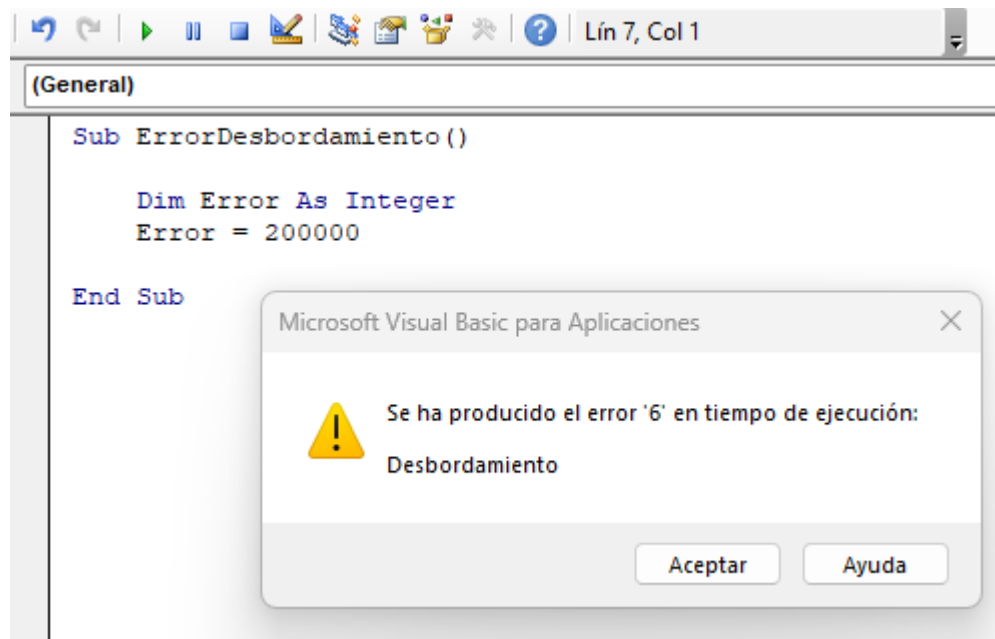
Dim MiVariable **As** String

Para declara una variable constante se debe realizar de la siguiente manera:

Const MiVariable **As** Integer=50

Errores frecuentes con variables

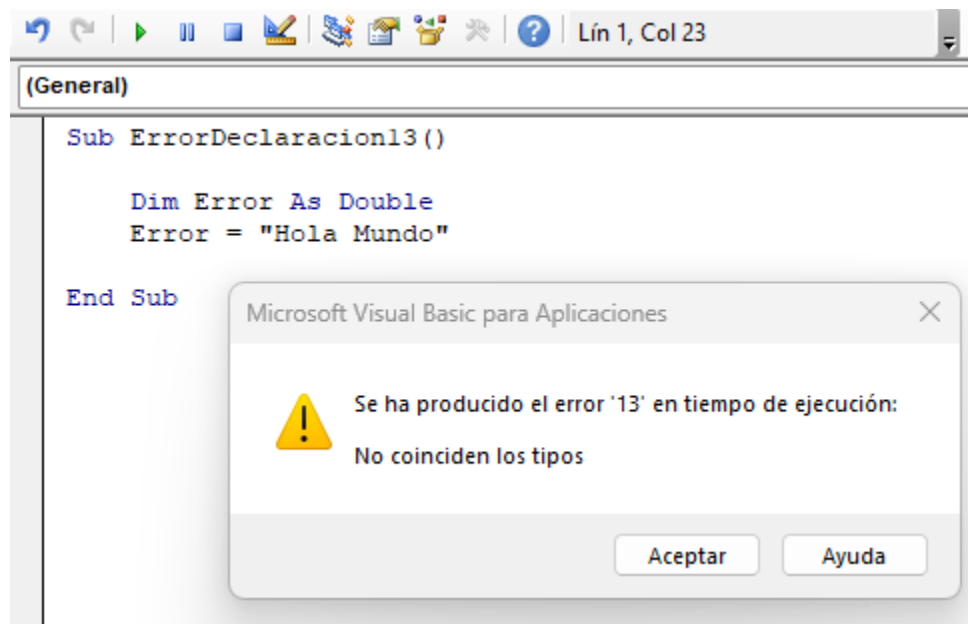
Error por desbordamiento. También conocido como error 6 en tiempo de ejecución y se genera cuando se trata de almacenar un valor que excede los límites que permite ciertos tipos de datos.



Las causas más comunes de este error se generan por la asignación, cálculo o conversión de un tipo de dato que supera los caracteres permitidos para que se puedan representar dentro de un rango de valores permitidos para para este tipo de variables.

Una posible solución es cambiar el tipo de dato que pueda almacenar dicha variable, como es doble a un float, el cual puede almacenar números más grandes.

Otro error común se denomina como 3D en VBA, el cual se presenta cuando no hay coincidencia entre la variable que se declara y la variable que se ejecuta, ya que cada variable en Excel posee diferentes características y propiedades; al no coincidir los tipos de variables lo asume como un error de ejecución.



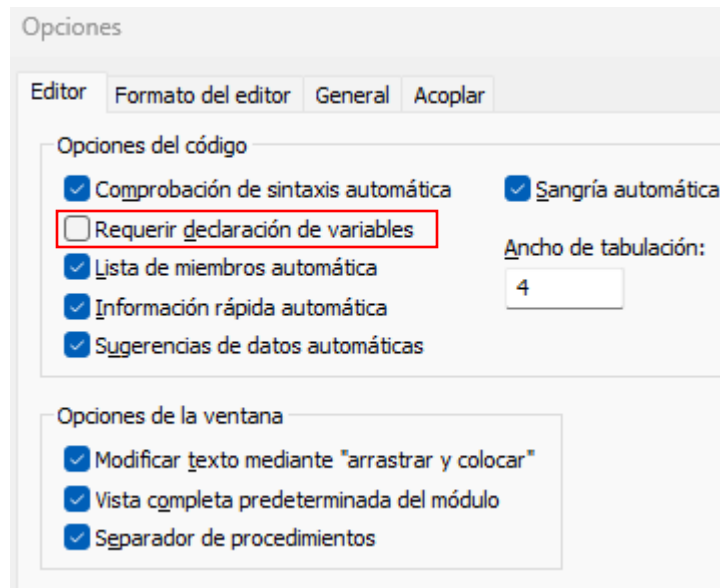
La solución para este tipo de error es revisar muy bien los tipos de datos que se están intentando procesar o corregir el tipo de dato que se quiere almacenar en la variable.

Opción Explicit

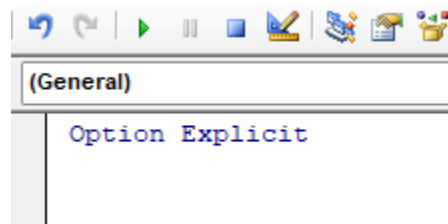
Es importante recordar que esta función permite determinar que de alguna manera cada una de las variables sean declaradas o no, en caso de estar activo y no declarar ninguna variable. En este caso, el editor va a generar un error al momento de ejecutar un proceso.

Recordemos que esta opción se puede habilitar desde:

El menú de VBA, herramientas, opciones, clic en habilitar "requerir declaración de variables".

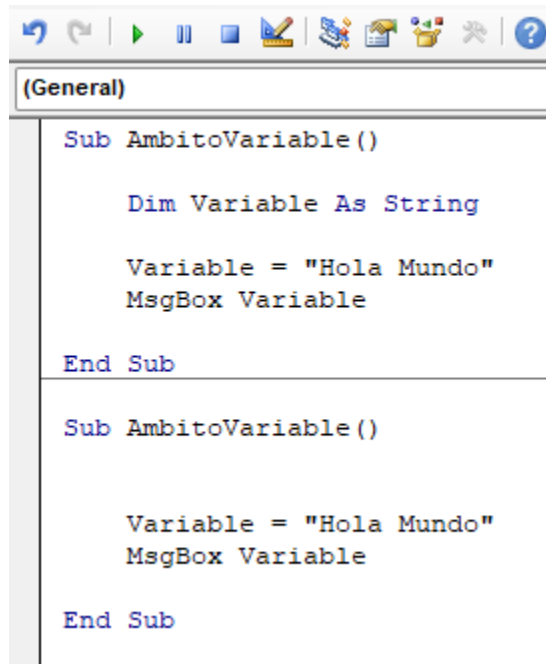


La abrir un nuevo módulo se puede observar que ya aparece esta opción habilitada.



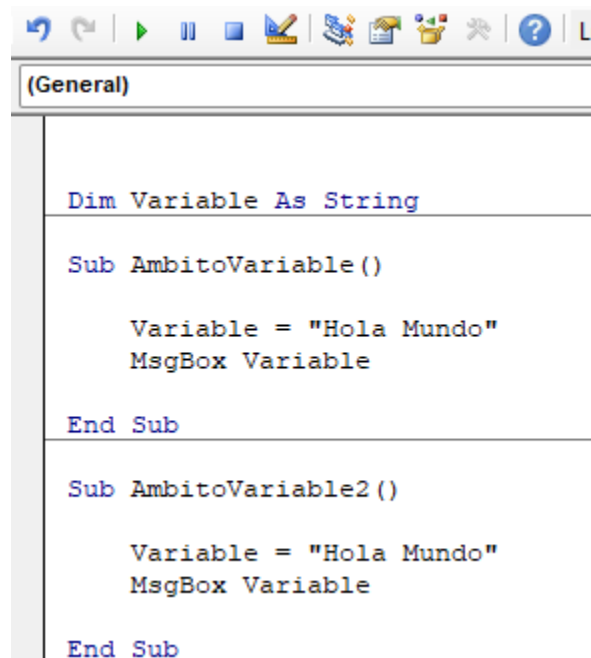
Ámbito de las variables

El ámbito de las variables hace referencia al alcance que esta variable tiene cuando se declara en VBA; en este caso existen tres tipos de alcance de procedimiento o de macro, un alcance de módulo y un alcance de proyecto, hablando en el ámbito de un procedimiento. Lo anterior se refiere a una variable definida a nivel de procedimiento en el que es declarada, es decir, al declarar una variable DIM, solo va a hacer reconocida en el procedimiento donde se encuentra.



```
Sub AmbitoVariable()  
  
    Dim Variable As String  
  
    Variable = "Hola Mundo"  
    MsgBox Variable  
  
End Sub  
  
Sub AmbitoVariable()  
  
    Variable = "Hola Mundo"  
    MsgBox Variable  
  
End Sub
```

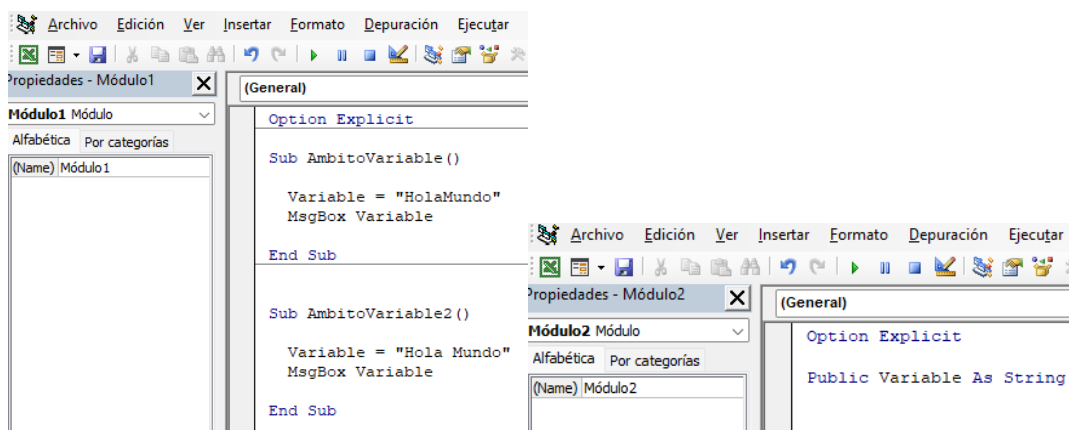
En el ámbito de módulo de un variable permite que dicha variable se ejecute en todos los procedimientos del módulo. En este caso el uso de la palabra clave DIM significa que esta variable va a hacer declarada a nivel de módulo ya que esta declarada por fuera del procedimiento.



```
Dim Variable As String  
  
Sub AmbitoVariable()  
  
    Variable = "Hola Mundo"  
    MsgBox Variable  
  
End Sub  
  
Sub AmbitoVariable2()  
  
    Variable = "Hola Mundo"  
    MsgBox Variable  
  
End Sub
```

El tercer ámbito que se refiere a nivel de proyecto en donde se va a tener un módulo, por ejemplo, un módulo donde se puede dedicar a la declaración de las diferentes variables que se vayan a utilizar en dicho proyecto y las cuales se pueden llamar/invocar desde diferentes procedimientos que se encuentran en diferentes módulos o formularios a través de la declaración de "Public

Variable As String”. En ese sentido, si esta variable se encuentra en otro módulo diferente se va a ejecutar sin problema.



Funciones propias de VBA*

Funcion MsgBox

La función MsgBox, la cual permite mostrar un mensaje dentro de un cuadro mientras que el usuario haga clic sobre ciertos botones provistos en el mismo mensaje, de esta manera se ejecutara alguna instrucción dependiendo el botón seleccionado.

Esta función cuenta con cinco argumentos básicos los cuales se presentan a continuación:

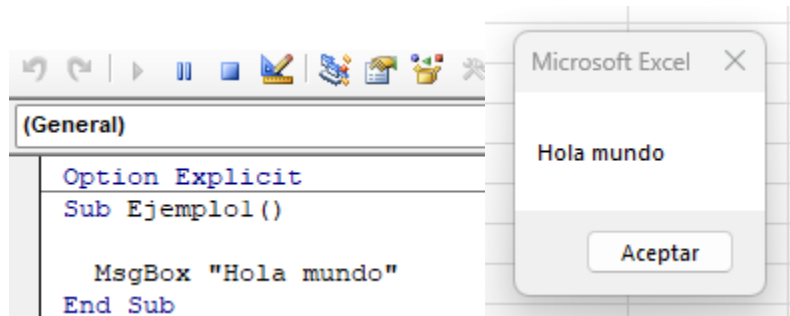
El primer caso, se trata del mensaje que debe tener la función MsgBox; aquí algunas consideraciones a la hora de utilizarlo.

Grupo	Constante	Descripción
Botones	vbOKOnly	Muestra el botón aceptar
	vbOKCancel	Muestra los botones de aceptar y cancelar
	vbAbortRetryIgnore	Muestra los botones de anular, reintentar y omitir
	vbYesNoCancel	Muestra los botones si, no y cancelar
	vbYesNo	Muestra los botones si y no
	vbRetryCancel	Muestra los botones de reintentar y cancelar
Tipo de Icono	vbCritical	Muestra icono de mensaje critico
	vbQuestion	Muestra icono de consulta de advertencia
	vbExclamation	Muestra icono mensaje de emergencia
	vbInformation	Muestra icono mensaje de informacion
Botón predeterminado	vbDefaultButton1	El primer botón es el predeterminado
	vbDefaultButton2	El segundo botón es el predeterminado
	vbDefaultButton3	El tercer botón es el predeterminado
	vbDefaultButton4	El cuarto botón es el predeterminado
Modalidad	vbApplicationModal	Se suspenden todas las aplicaciones hasta que el usuario responda al cuadro del mensaje
	vbSystemModal	

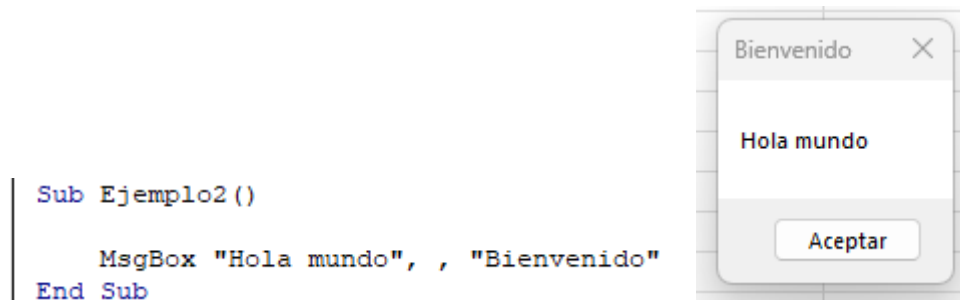
Alineación de texto y botón de ayuda	vbMsgBoxHelpButton	Muestra el botón de ayuda
	vbMsgBox	Especifica la ventana del cuadro de mensaje como ventana en el primer plano
	vbMsgBox	Texto alineado a la derecha
	vbMsgBox	Especifica que el texto debe aparecer para ser leído de derecha a izquierda en los sistemas árabe y hebreo.

Algunos ejemplos:

a) Mensaje básico



b) mensaje básico con título

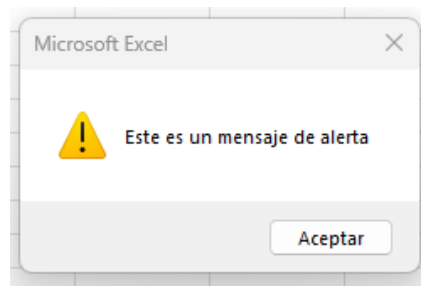


c) mensaje de advertencia

```

Sub Ejemplo3()
    MsgBox "Este es un mensaje de alerta", vbExclamation
End Sub

```

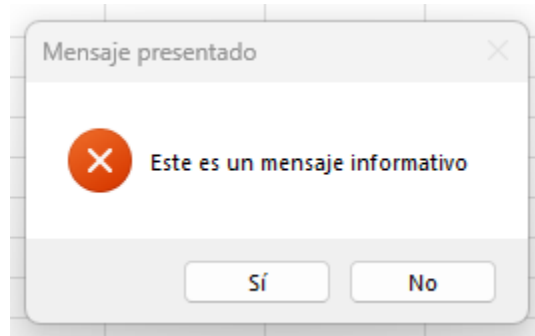


d) mensaje de informativo con opción de dos botones

```
Sub Ejemplo4()

    MsgBox "Este es un mensaje informativo", vbCritical + vbYesNo, "Mensaje presentado"

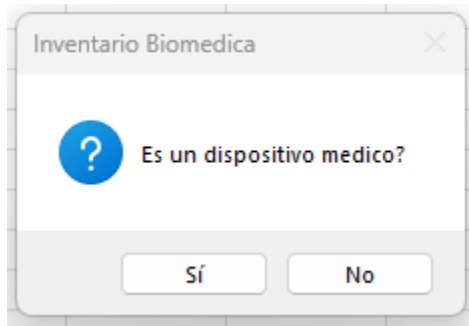
End Sub
```



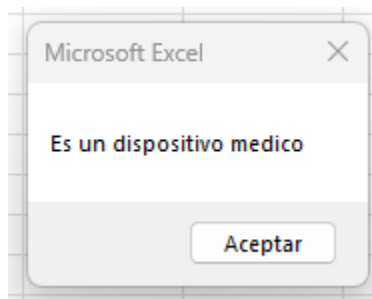
e) mensaje de verificación de dos opciones

```
Sub Ejemplo5()

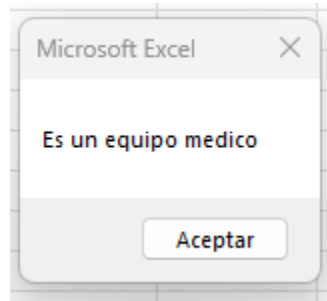
    If vbYes = MsgBox("Es un dispositivo medico?", vbQuestion + vbYesNo, "Inventario Biomedica") Then
        MsgBox "Es un dispositivo medico"
    Else
        MsgBox "Es un equipo medico"
    End If
End Sub
```



Si la respuesta es si,



Si la respuesta es no,

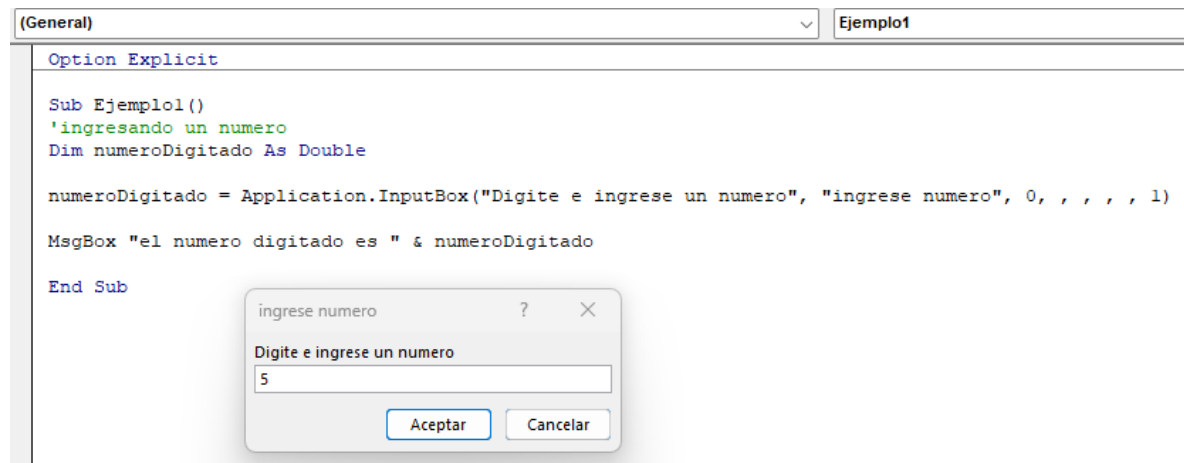


Función InputBox

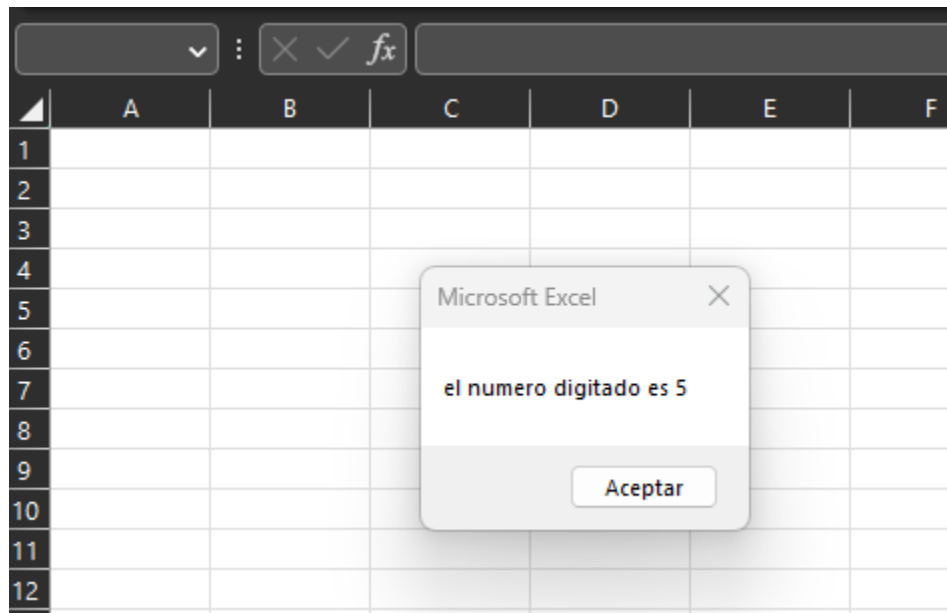
Es una función que permite mostrar un cuadro de dialogo para que el usuario pueda ingresar información; en lógica de programación se conoce como caja de mensajes en donde se lee un dato en particular y se asignaba a una variable.

Esta función es muy utilizada en Excel para interactuar con los usuarios o porque no, enviar o mostrar un mensaje o aviso de forma informativa.

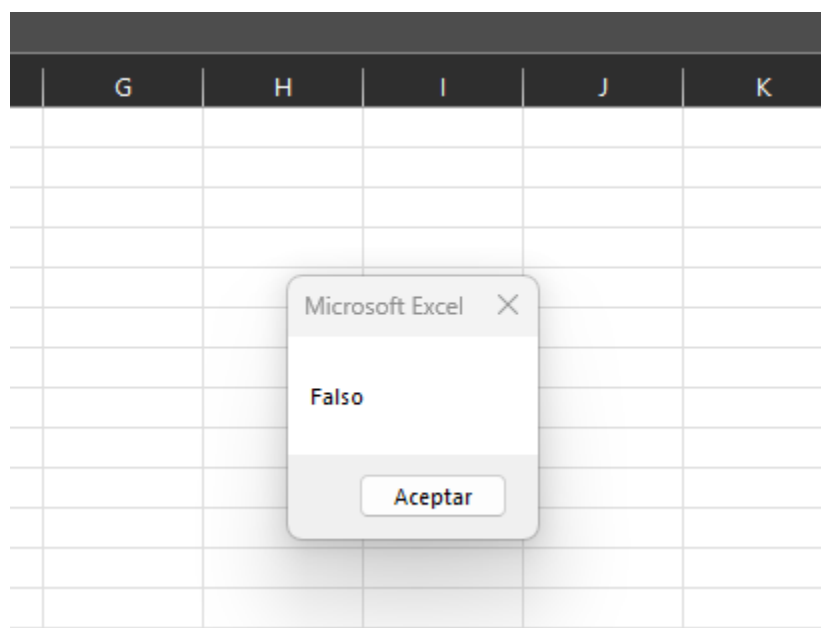
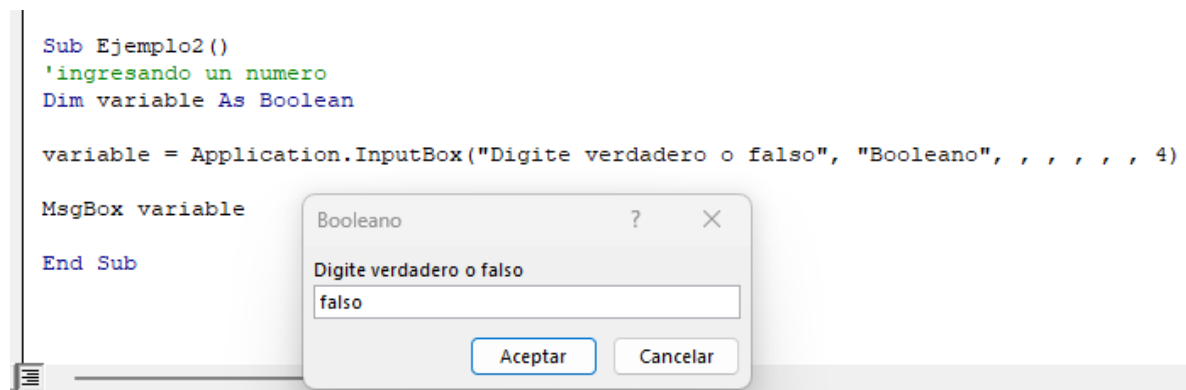
A continuación, un ejemplo:



Como resultado se obtiene un mensaje en la hoja de Excel,



Otro ejemplo con valores booleanos,



Funciones de información

Las funciones de información en VBA son utilizadas para verificar el tipo de dato que se esta utilizando por medio de una variable. En ese aspecto, existen cuatro funciones principales que se explicaran a continuación:

Error	Descripción
#N/D	En la formula o función existe algún valor que no se encuentra
#!Valorj	Se está utilizando un operando o argumento no valido, por ejemplo, texto en lugar de números
#!Refj	La fórmula se está refiriendo a una celda que no es válida, bien sea porque no existe, es decir al seleccionar por ejemplo una celda "a3000000" devuelve este error ya que dicha celda no existe en Excel.
#!Div/0j	Aparece cuando en una función se divide un número cualquiera entre cero o una celda vacía.
#!Numj	Indica que los valores numéricos no son válidos o el número es demasiado grande o pequeño para ser procesado y por ende representado por Excel.
#¿Nombre?	El texto de la fórmula que se ha digitado no es reconocido por Excel.
#!Nuloj	En una formula se hace referencia a dos rangos que no tienen intersección entre ellos.

Funciones de texto

Se utiliza para el manejo de cadenas de texto que por lo general se encuentran declaradas como tipo string. A continuación, se presentan algunas de las más importantes:

La primera función es conocida como Rigg, que permite obtener caracteres que se encuentran al lado derecho de una frase que se encuentra en una variable o en una celda de Excel. A continuación, un ejemplo de ello:

```
(General)

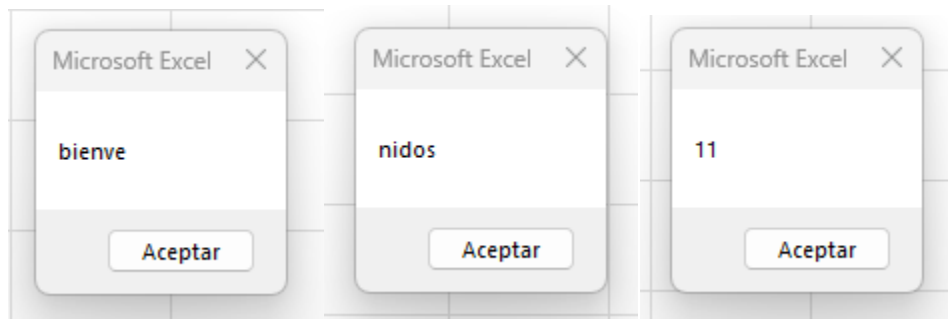
Sub Ejemplo_Funcion_Right_left_Len()

    Dim Texto As String
    Texto = "bienvenidos"

    MsgBox Left(Texto, 6)
    'muestra las primeras 6 letras
    MsgBox Right(Texto, 5)
    'muestra las ultimas 5 letras

    MsgBox Len(Texto)

End Sub
```

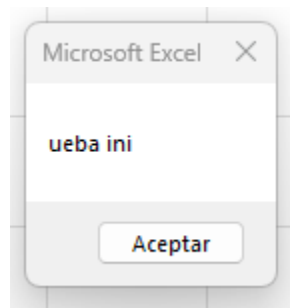


Otro ejemplo,

```
(General)

Sub Ejemplo_Funcion_Mid()
    Dim Texto As String
    Texto = "prueba inicial"

    MsgBox Mid(Texto, 3, 8)
    'extrae un rango de caracteres
End Sub
```



Funciones de fecha y hora

Las funciones de fecha y hora permiten la manipulación de variables que tienen datos de tipo fecha o en su defecto que se necesitan convertir en este tipo de datos. En este caso, dos de las principales funciones son Now y Date, las cuales permiten obtener la fecha y hora del pc donde se está ejecutando Excel.

Un ejemplo:

```
(General)

Sub Ejemplo_Fecha_Hora()

    Dim Fecha As Date
    Fecha = Now

    Debug.Print Year(Fecha) 'imprime año
    Debug.Print Month(Fecha) 'imprime numero de mes
    Debug.Print Day(Fecha)
    Debug.Print TimeValue(Fecha)
    Debug.Print Hour(Fecha)
    Debug.Print Minute(Fecha)
    Debug.Print Second(Fecha)

End Sub
```

En la ventana de Inmediato se puede observar:

```
Inmediato

13/01/2023
13/01/2023 8:00:30 p. m.
2023
1
13
8:03:58 p. m.
20
```

Otras funciones utilizadas son:

```
(General)

Sub Ejemplo3()

    Dim Fecha As Date
    Fecha = Now
    'presentacion de fecha y formato
    Debug.Print FormatDateTime(Fecha, vbGeneralDate)
    Debug.Print FormatDateTime(Fecha, vbLongDate)
    Debug.Print FormatDateTime(Fecha, vbLongTime)
    Debug.Print FormatDateTime(Fecha, vbShortDate)
    Debug.Print FormatDateTime(Fecha, vbShortTime)

End Sub
```

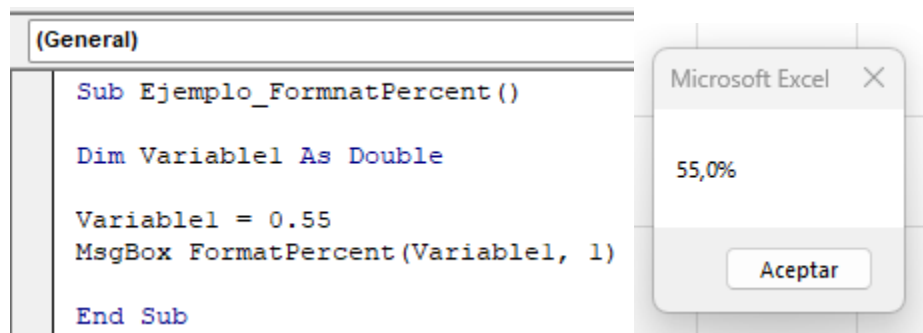
```
Inmediato

13/01/2023 8:19:48 p. m.
viernes, 13 de enero de 2023
8:19:48 p. m.
13/01/2023
20:19
```

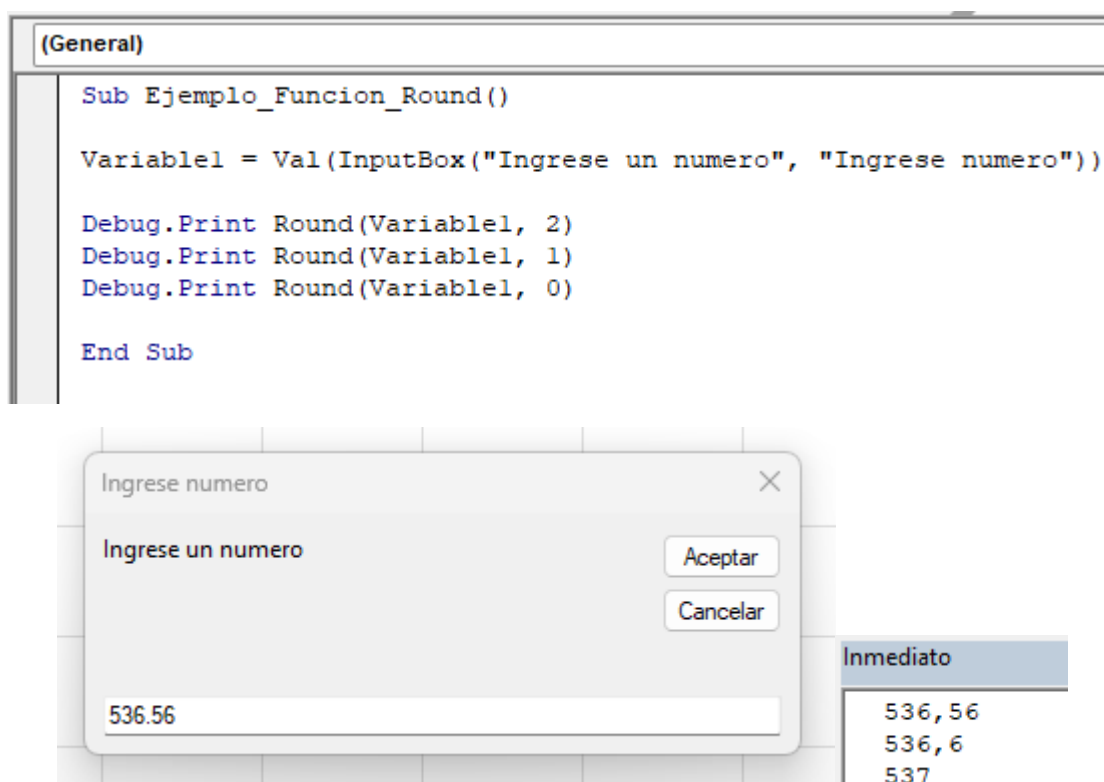
Funciones matemáticas

Algunas de las funciones matemáticas más relevantes se encuentran a continuación:

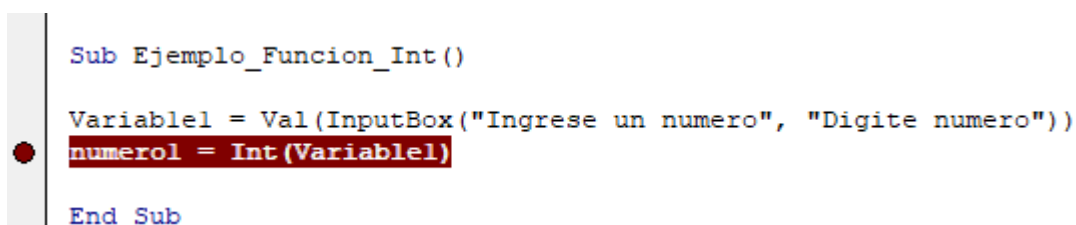
Ejemplo de Formnat_Percent (formato de porcentaje)



Otro ejemplo de cómo redondear cifras,



Otro ejemplo que devuelve la parte entera de un numero:



Digite numero

Ingrese un numero

Aceptar

Cancelar

285.87

Locales		
VBAProject.ThisWorkbook.Ejemplo_Funcion_Int		
Expresión	Valor	Tipo
Me		ThisWorkbook/ThisWorkbook
Variab1e1	285,87	Variant/Double
numero1	Vacio	Variant/Empty

Funciones de conversión

```

(General)

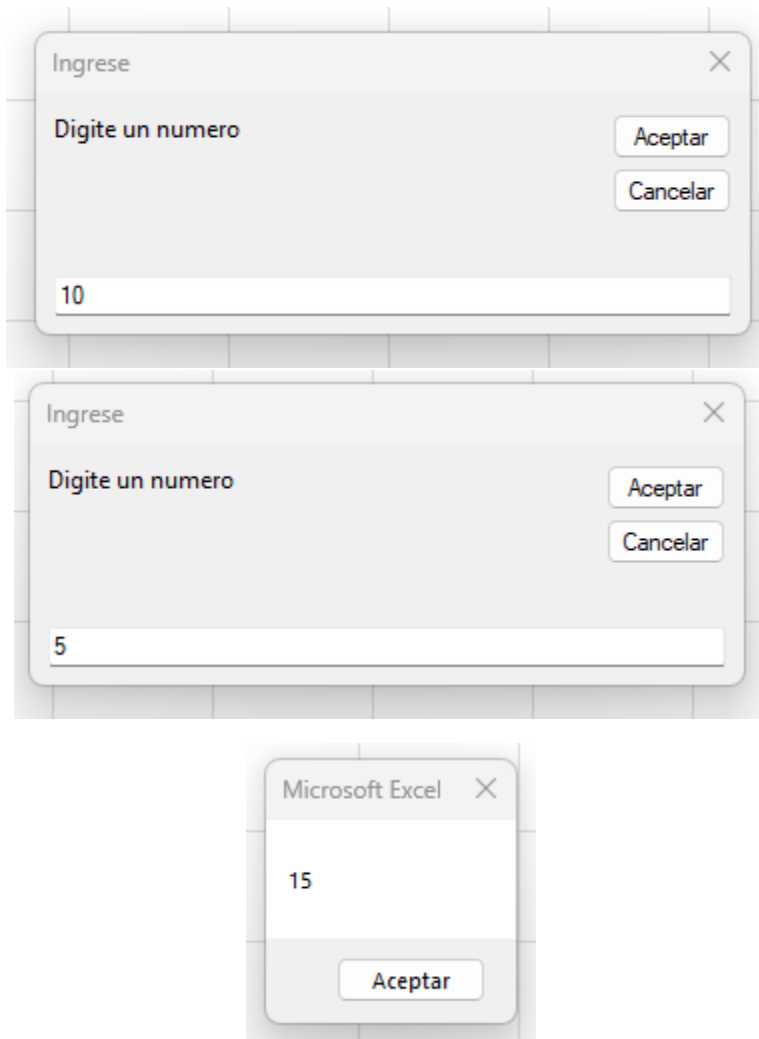
Dim Variable1 As String
Dim Variable2 As String
Dim Variable3 As Long
Dim Suma As Double

Sub Funcion_Val()

    Variable1 = InputBox("Digite un numero", "Ingrese")
    Variable2 = InputBox("Digite un numero", "Ingrese")
    ' Val convierte el formato en un valor numerico
    MsgBox Val(Variable1) + Val(Variable2)

End Sub

```



Ejemplo de CInt convierte un valor a número entero y Cdbl convierte un string a un valor double.

Un ejemplo:

```
Sub Funcion_CInt_Cdbl()  
    Variable1 = InputBox("Digite un numero", "Ingreso")  
    Variable2 = InputBox("Digite un numero", "Ingreso")  
    ' CInt convierte valores menores a 32000  
    MsgBox CInt(Variable1) + CInt(Variable2)  
End Sub
```



```

Sub Funcion_CInt_Cdbl()

    Variable1 = InputBox("Digite un numero", "Ingrese")
    Variable2 = InputBox("Digite un numero", "Ingrese")
    ' CInt convierte valores mayores a 32000
    MsgBox CDb1(Variable1) + CDb1(Variable2)

End Sub

```

Otro ejemplo es Cdate que convierte un valor a formato fecha

```

Sub Funcion_Cdate()

    Variable3 = 44945
    Debug.Print CDate(Variable3)
    ' convierte un valor a fecha

End Sub

```

Operadores matemáticos

Son aquello que se utilizan para realizar operaciones como suma, resta, multiplicación y división, entre otros más. Sin embargo, es necesario tener en cuenta la regla de prioridades claves a la hora de realizar operaciones como tal.

Los operadores lógicos son utilizados en informática y sirven para evaluar el conocimiento de dos o más condiciones que se encuentran en una estructura condicional en VBA. Estos Operadores son:

Operador Y (AND)

Condición 1	Condición 2	Resultado
Verdadero	Verdadero	Verdadero
Verdadero	Falso	Falso
Falso	Verdadero	Falso
Falso	Falso	Falso

Operador O (Or)

Condición 1	Condición 2	Resultado
Verdadero	Verdadero	Verdadero
Verdadero	Falso	Verdadero
Falso	Verdadero	Verdadero
Falso	Falso	Falso

Operador No (Not)

Condición	Resultado
Verdadero	Falso
Falso	Verdadero

Operadores relacionales

Son símbolos que se utilizan para comparar dos valores diferentes dentro de una estructura condicional. Si la comparación es correcta su resultado será verdadera; de lo contrario su resultado será Falso.

Existen 6 operadores relacionales:

Operador	Nombre
<	Mayor que
>	Menor que
=	Igual a
<>	Diferente a
<=	Menor o igual que
>=	Mayor o igual que

Operadores de concatenación

Se utilizan principalmente para unir uno o más valores o para unir cadenas de texto; por lo general se hace uso de los caracteres & o +.

Un ejemplo de ello:

Variable: "Hola" & "Mundo" = HolaMundo

Variable: "Hola" & " " & "Mundo" = Hola Mundo

Condicionales

If – End If (condicional simple)

La condición If sirve para comprobar una o varias condiciones en una expresión, es decir, evalúa una condición bajo una prueba lógica donde se obtendrán dos posibles resultados: Verdadero o Falso.

Un ejemplo:

```

Sub NumeroMayor()
    Dim Varibale1 As Double, Varibale2 As Double

    Variable1 = InputBox("Digite primer numero", "Ingrese un numero")
    Variable2 = InputBox("Digite segundo numero", "Ingrese un numero")

    If Variable1 > Variable2 Then
        MsgBox Variable1
    Else
        MsgBox Variable2
    End If

End Sub

```

Condicionales anidados If – If Else

Se refieren al uso de uno o más condiciones dentro de otra, con el objetivo de evaluar posibles respuestas. A continuación, un ejemplo:

```

Sub EvaluacionEquipos()

    Dim Mantenimiento As Integer
    Mantenimiento = InputBox("Digite el tiempo de vida util", "ingrese un valor")

    If Mantenimiento >= 10 Then
        MsgBox "necesita mantenimiento preventivo"
    Else
        If Mantenimiento >= 5 Then
            MsgBox "necesita inspeccion"
        Else
            If Mantenimiento >= 0 Then
                MsgBox "Equipo ok"
            Else
                MsgBox "no es valido el valor ingresado"
            End If
        End If
    End If

End Sub

```

Condicionales compuestos If + And + Or

Este tipo de condiciones compuestas evalúan más de una condición en una sola expresión, donde se utiliza los operadores And, &, Or (tablas de verdad).

Un ejemplo de ello:

```

Sub EvaluacionEquipos2()

    Dim Mantenimiento As Double
    Mantenimiento = InputBox("Digite el tiempo de vida util")

    If Mantenimiento >= 5 Then
        MsgBox ("Necesita mantenimiento (se encuentra entre 6 y 10 años)")
    Else
        MsgBox ("Equipo en buen estado (se encuentra entre 0 y 5 años)")
    End If

End Sub

```

Condicionales de selección multiple Select Case

Se utiliza cuando se desea o se necesita aplicar muchas condiciones a un valor o variable. A continuación, un ejemplo de ello:

```

Sub SeleccionCategoria()
    Dim ServicioEquipo As Integer
    ServicioEquipo = InputBox("Ingrese un valor de servicio:" & vbCrLf & "1 UCI, 2 Urgencias" & _
        "3 Salas Op, 4 Neonatos, 5 Triage")

    Select Case ServicioEquipo
        Case 1:
            MsgBox "UCI"
        Case 2:
            MsgBox "Urgencias"
        Case 3:
            MsgBox "Salas Op"
        Case 4:
            MsgBox "Neonatos"
        Case 5:
            MsgBox "Triage"
        Case Else
            MsgBox "Valor ingresado invalido"
    End Select

End Sub

```

Estructuras repetitivas

Estas estructuras permiten la repetición de una o varias secuencias de instrucciones, hasta la condición o prueba lógica deje de ser verdadera.

Para el ciclo para (for) hasta que la variable llegue a la cantidad de repeticiones que se han definido.

El ciclo **for next**, se utiliza cuando se conoce la cantidad de operaciones o las veces a repetir un conjunto de instrucciones.

Do while do, el cual se ejecuta cuando una condición cualquiera sea verdadero.

De otro lado el ciclo **do loop While**, se ejecuta mientras que una condición cualquiera sea verdadero.

El ciclo **for each** se utiliza para recorrer un conjunto de objetos del mismo tipo.

Ciclo For-Next

Este ciclo va acompañado de una variable contadora desde un número inicial a uno final, estos límites se definen de manera independiente.

Un ejemplo de ello:

```
Sub EjemploFor()  
  
    Dim NumeroMultiplicar As Double  
  
    NumeroMultiplicar = InputBox("Digite un numero")  
    For i = 1 To 10  
        Debug.Print i & "x" & NumeroMultiplicar & "=" & i * NumeroMultiplicar  
    Next i  
  
End Sub  
}
```

Ciclo Do While – Loop

Este condicional se ejecuta mientras la condición sea verdadera en caso contrario se dejara de ejecutar las condiciones que se encuentren dentro del ciclo o en el caso de no continuar más líneas de código se terminara la ejecución.

```
Sub EvaluacionValor()  
  
    Dim Valor As String  
  
    Valor = InputBox("digite un numero")  
    Do While Valor <> "p"  
  
        If CDBl(Valor) = 0 Then  
            MsgBox "el numero digitado es cero"  
        Else  
            If CDBl(Valor) > 0 Then  
                MsgBox "el numero digitado es positivo"  
            Else  
                MsgBox "el numero digitado es negativo"  
            End If  
        End If  
        Valor = InputBox("Ingrese otro numero")  
    Loop  
  
    MsgBox "Se ingreso la letra p y se finalizo la secuencia"  
  
End Sub
```

Ciclo Do – Loop While

Se utiliza para ya que la primera vez que se ejecuta las instrucciones que se encuentran dentro del ciclo, luego evalúa la condición y el caso que retorne verdadero, seguirá ejecutándose nuevamente las instrucciones que se encuentran dentro del ciclo.

```
Sub NumeroSuperior()  
    Dim NumeroMayor As Double  
    Dim NumeroLeido As Double  
  
    NumeroMayor = 0  
  
    Do  
        NumeroLeido = InputBox("Digite un numero", "Ingrese Numero")  
        If NumeroLeido > NumeroMayor Then  
            NumeroMayor = NumeroLeido  
        End If  
    Loop While NumeroLeido > 0  
    MsgBox ("El numero mayor es " & NumeroMayor)  
End Sub
```